Learned Step Size Quantization

26/03/20

Step (a.k.a. Scale)

Quantization-Aware Training (QAT)

[기존 문제]

To date, most approaches for training low precision networks have employed uniform quantizers, which can be configured by a single step size parameter (the width of a quantization bin), though more complex nonuniform mappings have been considered (Polino et al., 2018). Early work with low precision deep networks used a simple fixed configuration for the quantizer (Hubara et al., 2016; Esser et al., 2016), while starting with Rastegari et al. (2016), later work focused on fitting the quantizer to the data, either based on statistics of the data distribution (Li & Liu, 2016; Zhou et al., 2016; Cai et al., 2017; McKinstry et al., 2018) or seeking to minimize quantization error during training (Choi et al., 2018c; Zhang et al., 2018). Most recently, work has focused on using backpropagation with

While attractive for their simplicity, fixed mapping schemes based on user settings place no guarantees on optimizing network performance, and quantization error minimization schemes might perfectly minimize quantization error and yet still be non optimal if a different quantization mapping actually minimizes task error. Learning the quantization mapping by seeking to minimize task loss is appealing to us as it directly seeks to improve on the metric of interest. However, as the quantizer itself is discontinuous, such an approach requires approximating its gradient, which existing methods have done in a relatively coarse manner that ignore the impact of transitions between quantized states (Choi et al., 2018b;a; Jung et al., 2018).

1. Task 성능과의 괴리: Quantization Error를 줄이는 데에만 집중/Fixed & Single
2. Gradient 근사 및 transition 부근에서 고려 x
3. Gradient update 불균형 (scale vs. weight)

[Contribution]

Here, we introduce a new way to learn the quantization mapping for each layer in a deep network, *Learned Step Size Quantization* (LSQ), that improves on prior efforts with two key contributions. First, we provide a simple way to approximate the gradient to the quantizer step size that is sensitive to quantized state transitions, arguably providing for finer grained optimization when learning the step size as a model parameter. Second, we propose a simple heuristic to bring the magnitude of step size updates into better balance with weight updates, which we show improves convergence. The overall approach is usable for quantizing both activations and weights, and works with existing methods for backpropagation and stochastic gradient descent. Using LSQ to train several network architectures on

1. Learnable Step(=Scale) via original task loss
2. Transition 부근에 민감한 new gradient approximation
3. Gradient Scaling을 통해 step과 weight가 균형있게 update 되도록 함

We consider deep networks that operate at inference time using low precision integer operations for computations in convolution and fully connected layers, requiring quantization of the weights and activations these layers operate on. Given data to quantize v, quantizer step size s, the number of positive and negative quantization levels Q_P and Q_N, respectively, we define a quantizer that computes $\bar{v}$, a quantized and integer scaled representation of the data, and $\hat{v}$, a quantized representation of the data at the same scale as v:

$$\bar{v} = \lfloor clip(v/s, -Q_N, Q_P) \rceil, \tag{1}$$

$$\hat{v} = \bar{v} \times s. \tag{2}$$

Here, $clip(z, r_1, r_2)$ returns z with values below r_1 set to r_1 and values above r_2 set to r_2, and $\lfloor z \rceil$ rounds z to the nearest integer. Given an encoding with b bits, for unsigned data (activations) $Q_N = 0$ and $Q_P = 2^b - 1$ and for signed data (weights) $Q_N = 2^{b-1}$ and $Q_P = 2^{b-1} - 1$.

For inference, $\bar{w}$ and $\bar{x}$ values can be used as input to low precision integer matrix multiplication units underlying convolution or fully connected layers, and the output of such layers then rescaled by the step size using a relatively low cost high precision scalar-tensor multiplication, a step that can potentially be algebraically merged with other operations such as batch normalization (Figure 1).

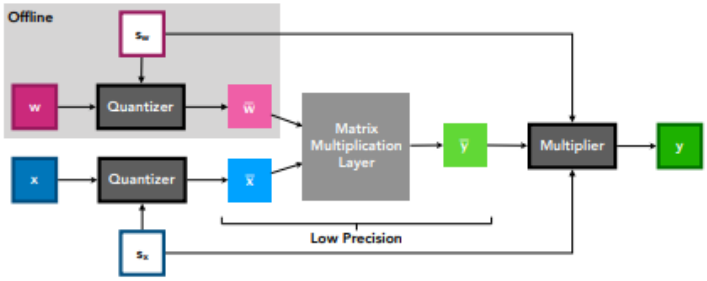
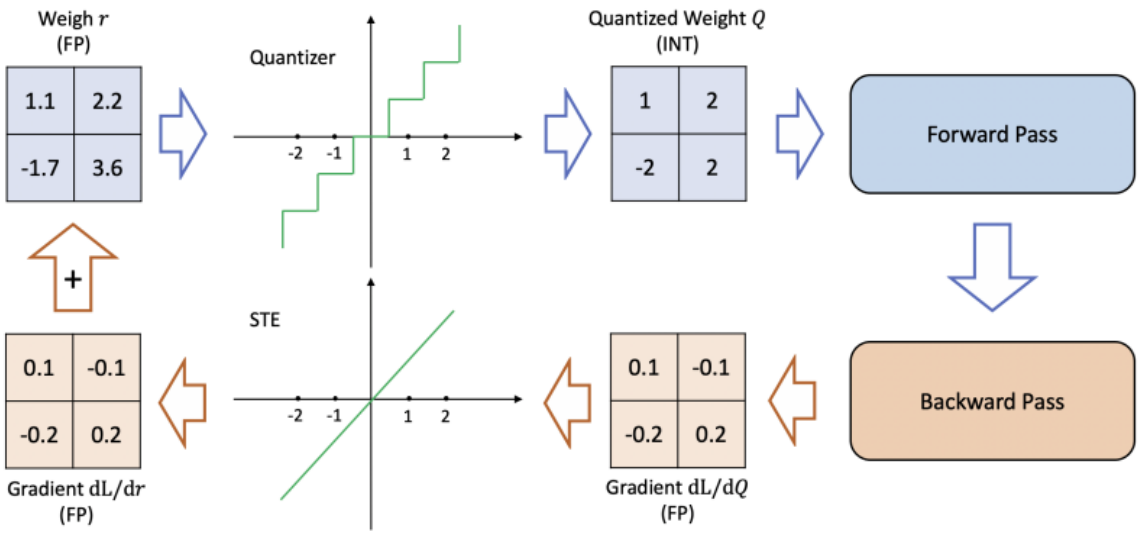


Figure 1: Computation of a low precision convolution or fully connected layer, as envisioned here.

학습을 위해선, backprop을 통한 가중치 update가 필요.
하지만, clip 함수는 아래와 같이 미분 불가능.
기존 방식(STE)는 아래와 같이 평균을 통해 기울기 근사.



[제안 Method]

2.1 STEP SIZE GRADIENT

LSQ provides a means to learn s based on the training loss by introducing the following gradient through the quantizer to the step size parameter:

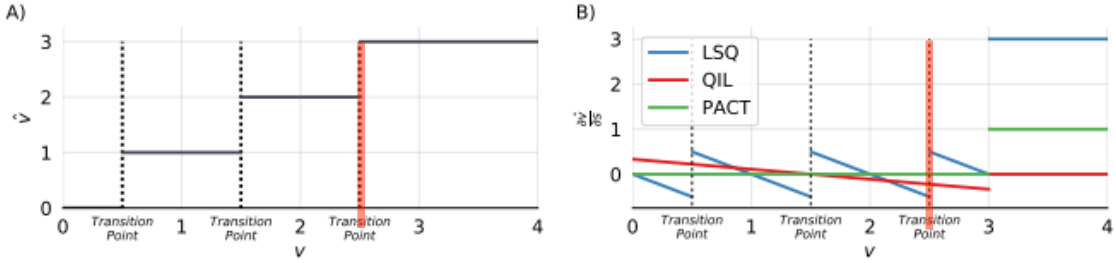
$$\frac{\partial \hat{v}}{\partial s} = \begin{cases} -v/s + \lfloor v/s \rceil & \text{if } -Q_N < v/s < Q_P \\ -Q_N & \text{if } v/s \leq -Q_N \\ Q_P & \text{if } v/s \geq Q_P \end{cases} \tag{3}$$

$$\frac{\partial \hat{v}}{\partial s} = \frac{\partial(\bar{v} \times s)}{\partial s} \quad (\because \hat{v} = \bar{v} \times s)$$

$$= s \times \frac{\partial \bar{v}}{\partial s} + \bar{v} \times \frac{\partial s}{\partial s} = s \times \frac{\partial \lfloor v/s \rceil}{\partial s} + \bar{v}$$

$$= s \times \frac{\partial \frac{v}{s}}{\partial s} + \bar{v} = s \times \frac{\partial (v \cdot s^{-1})}{\partial s} + \bar{v}$$

$$= s \times (-1) \times v \cdot s^{-2} + \bar{v} = \boxed{-\frac{v}{s} + \lfloor v/s \rceil}$$



closer a given v is to a quantization transition point, the more likely it is to change its quantization bin ($\bar{v}$) as a result of a learned update to s (since a smaller change in s is required), thereby resulting in a large jump in $\hat{v}$. Thus, we would expect $\partial\hat{v}/\partial s$ to increase as the distance from v to a transition point decreases, and indeed we observe this relationship in the LSQ gradient. It is appealing that this gradient naturally falls out of our simple quantizer formulation and use of the straight through estimator for the round function.

For this work, each layer of weights and each layer of activations has a distinct step size, represented as an fp32 value, initialized to $2\langle|v|\rangle/\sqrt{Q_P}$, computed on either the initial weights values or the first batch of activations, respectively.

Transition point(=Clip에 의해 크게 변하는 지점)에서는 조금만 값이 변해도 다른 양자화 bin 값을 jump 하기 때문에, quantization 전후 간 큰 차이가 발생

이를 위해, LSQ에서는 위의 gradient 식을 통해 값이 바뀔 위험이 큰 transition point 부근의 상태를 step size 학습에 더 반영하기 위함.

학습 대상: 모든 각 layer들의 weight와 activation에 대응하는 각각의 step을 학습.

2.2 STEP SIZE GRADIENT SCALE

It has been shown that good convergence is achieved during training where the ratio of average update magnitude to average parameter magnitude is approximately the same for all weight layers in a network (You et al., 2017). Once learning rate has been properly set, this helps to ensure that all updates are neither so large as to lead to repeated overshooting of local minima, nor so small as to lead to unnecessarily long convergence time. Extending this reasoning, we consider that each step size should also have its update magnitude to parameter magnitude proportioned similarly to that of weights. Thus, for a network trained on some loss function L, the ratio

$$R = \frac{\nabla_s L}{s} \Big/ \frac{\|\nabla_w L\|}{\|w\|} \tag{4}$$

should on average be near 1, where $\|z\|$ denotes the l_2-norm of z. However, we expect the step size parameter to be smaller as precision increases (because the data is quantized more finely), and step size updates to be larger as the number of quantized items increases (because more items are summed across when computing its gradient). To correct for this, we multiply the step size loss by a gradient scale, g, where for weight step size $g = 1/\sqrt{N_W Q_P}$ and for activation step size $g = 1/\sqrt{N_F Q_P}$, where N_W is the number of weights in a layer and N_f is the number of features in a layer. In section 3.4 we demonstrate that this improves trained accuracy, and we provide reasoning behind the specific scales chosen in the Section A of the Appendix.

R = 1 이 되도록, 즉 step과 weight가 균형 있게 update 되도록 하기 위함.

이를 위해 각각의 weight와 activation step update에 대해 gradient scale g를 곱함.

- 주황 밑줄

scaling factor to be defined, the clipping range $[\alpha, \beta]$ should first be determined. The process of choosing the clipping range is often referred to as *calibration*.

$S = \frac{\beta - \alpha}{2^b - 1}$ (b: bit width)

Precision increase(=b increase): 분모 ↑ S ↓

Quantized items increase(=alpha ~ beta 구간 ↑): 분자 ↑ S ↑

[Experimental Results]

1. **ImageNet에 대한 정량 결과**

Network	Method	Top-1 Accuracy @ Precision 2	3	4	8	Top-5 Accuracy @ Precision 2	3	4	8
ResNet-18		*Full precision: 70.5*				*Full precision: 89.6*			
	LSQ (Ours)	**67.6**	**70.2**	**71.1**	**71.1**	**87.6**	**89.4**	**90.0**	**90.1**
	QIL	65.7	69.2	70.1					
	FAQ			69.8	70.0			89.1	89.3
	LQ-Nets	64.9	68.2	69.3		85.9	87.9	88.8	
	PACT	64.4	68.1	69.2		85.6	88.2	89.0	
	NICE		67.7	69.8			87.9	89.21	
	Regularization	61.7		67.3	68.1	84.4		87.9	88.2
ResNet-34		*Full precision: 74.1*				*Full precision: 91.8*			
	LSQ (Ours)	**71.6**	**73.4**	**74.1**	**74.1**	**90.3**	**91.4**	**91.7**	**91.8**
	QIL	70.6	73.1	73.7					
	LQ-Nets	69.8	71.9			89.1	90.2		
	NICE		71.7	73.5			90.8	91.4	
	FAQ			73.3	73.7			91.3	91.6
ResNet-50		*Full precision: 76.9*				*Full precision: 93.4*			
	LSQ (Ours)	**73.7**	**75.8**	**76.7**	**76.8**	**91.5**	**92.7**	93.2	**93.4**
	PACT	72.2	75.3	76.5		90.5	92.6	93.2	
	NICE		75.1	76.5			92.3	**93.3**	
	FAQ			76.3	76.5			92.9	93.1
	LQ-Nets	71.5	74.2	75.1		90.3	91.6	92.4	
ResNet-101		*Full precision: 78.2*				*Full precision: 94.1*			
	LSQ (Ours)	**76.1**	**77.5**	**78.3**	**78.1**	**92.8**	**93.6**	**94.0**	**94.0**
ResNet-152		*Full precision: 78.9*				*Full precision: 94.3*			
	LSQ (Ours)	**76.9**	**78.2**	**78.5**	**78.5**	**93.2**	**93.9**	**94.1**	**94.2**
	FAQ			78.4	**78.5**			**94.1**	94.1
VGG-16bn		*Full precision: 73.4*				*Full precision: 91.5*			
	LSQ (Ours)	**71.4**	**73.4**	**74.0**	73.5	**90.4**	**91.5**	**92.0**	**91.6**
	FAQ			73.9	**73.7**			91.7	**91.6**
Squeeze Next-23-2x		*Full precision: 67.3*				*Full precision: 87.8*			
	LSQ (Ours)	**53.3**	**63.7**	**67.4**	**67.0**	**77.5**	**85.4**	**87.8**	**87.7**

2. **Weight Decay**

Table 2: ResNet-18 top-1 accuracy for various weight decay values.

Weight Decay	2-bit	3-bit	4-bit	8-bit
10^{-4}	66.9	70.1	**71.0**	**71.1**
0.5×10^{-4}	67.3	**70.2**	70.9	71.1
0.25×10^{-4}	**67.6**	70.0	70.9	71.0
0.125×10^{-4}	67.4	66.9	70.8	71.0

Precision ↓ : overfit ↓ 따라서, regularization도 줄여야 된다.

3. Gradient Scaling

Precision이 커질수록 step과 weight 간의 학습 불균형이 심해지는 경향 존재.

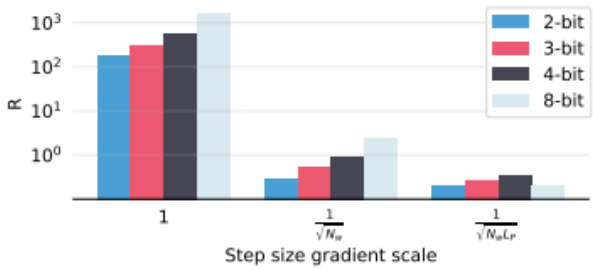


Figure 4: Relative parameter update magnitudes given different step size gradient scales. A gradient scale of $1/N_W Q_P$ better balances relative step size and weight gradient magnitudes (right vs. left).

Left: Gradient Scaling g를 곱하지 않았을 때 R 값이 100에 가까움. 이는 기존 R 식을 참고할 때 step에만 과도하게 학습되는 것을 알 수 있음.

Right: 논문에서 제시하는 Gradient Scaling을 적용하였을 때, R=1에 가까워져 step과 weight가 균형 있게 학습되는 것을 알 수 있음.

Table 3: Top-1 accuracy for various gradient scale values and learning rates for 2-bit ResNet-18.

Gradient scale	Learning Rate	Accuracy
$\mathbf{1/\sqrt{NQ_P}}$	**0.01**	**67.6**
$1/\sqrt{N}$	0.01	67.3
1	0.01	Did not converge
1	0.0001	64.2
$10/\sqrt{NQ_P}$	0.01	67.4
$1/10\sqrt{NQ_P}$	0.01	67.3

4. No Quantization Error Loss

For a 2-bit ResNet-18 model we found $\hat{s} = 0.949 \pm 0.206$ for activations and $\hat{s} = 0.025 \pm 0.019$ for weights (mean $\pm$ standard deviation). The percent absolute difference between $\hat{s}$ and the value of s that minimizes quantization error, averaged across activation layers was 50% for mean absolute error, 63% for mean square error, and 64% for Kullback-Leibler divergence, and averaged across weight layers, was 47% for mean absolute error, 28% for mean square error, and 46% for Kullback-Leibler divergence. This indicates that LSQ learns a solution that does not in fact minimize quantization error. As LSQ achieves better accuracy than approaches that directly seek to minimize quantization error, this suggests that simply fitting a quantizer to its corresponding data distribution may not be optimal for task performance.

Quantization Error → Task Error